



Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir
M.V. College
Of Science & Commerce

OS Practical No. 6 [10-08-2026]

Aim:

CPU Scheduling Algorithms (Part 2) – Round Robin Implement Round Robin scheduling with configurable time quantum. Compare with FCFS: fairness, turnaround, and response time. Track context switches and improve queue management.

4. Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given Time Quantum = 2 ms. Calculate average turn around time and average waiting time. For the same set of processes, schedule them using FCFS.

Process	Arrival Time	Burst Time
P1	0 ms	5 ms
P2	4 ms	2 ms
P3	5 ms	4 ms

CODE:

```
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.rem = bt
        self.ct = 0
        self.tat = 0
        self.wt = 0

def print_result(title, procs):
    print(f"\n--- {title} ---")
    print("PID\tAT\tBT\tCT\tTAT\tWT")
    total_tat = total_wt = 0
    for p in procs:
        print(f"P{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
        total_tat += p.tat
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
total_wt += p.wt
n = len(procs)
print(f"Average TAT = {total_tat / n:.2f} ms")
print(f"Average WT = {total_wt / n:.2f} ms")
```

```
def fcfs(procs):
    time = 0
    for p in procs:
        if time < p.at:
            time = p.at
        time += p.bt
        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt
    print_result("FCFS", procs)
```

```
def round_robin(procs, quantum):
    n = len(procs)
    queue = []
    arrived = [False] * n
    time = 0
    done = 0

    # enqueue processes already arrived at time 0
    for i, p in enumerate(procs):
        if p.at <= time and not arrived[i]:
            queue.append(i)
            arrived[i] = True

    while done < n:
        if not queue:
            # jump to the next process that hasn't arrived yet
            next_i = min((i for i in range(n) if not arrived[i]), key=lambda i: procs[i].at)
            time = procs[next_i].at
            queue.append(next_i)
            arrived[next_i] = True
            continue

        idx = queue.pop(0)
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
p = procs[idx]
```

```
slice_time = min(p.rem, quantum)
```

```
time += slice_time
```

```
p.rem -= slice_time
```

```
for i, q in enumerate(procs):
```

```
    if q.at <= time and not arrived[i]:
```

```
        queue.append(i)
```

```
        arrived[i] = True
```

```
if p.rem > 0:
```

```
    queue.append(idx)
```

```
else:
```

```
    p.ct = time
```

```
    p.tat = p.ct - p.at
```

```
    p.wt = p.tat - p.bt
```

```
    done += 1
```

```
print_result(f"Round Robin (Quantum = {quantum} ms)", procs)
```

```
if __name__ == "__main__":
```

```
    quantum = 2
```

```
p1 = [Process(1, 0, 5), Process(2, 4, 2), Process(3, 5, 4)]
```

```
p2 = [Process(1, 0, 5), Process(2, 4, 2), Process(3, 5, 4)]
```

```
round_robin(p1, quantum)
```

```
fcfs(p2)
```

OUTPUT:

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's Sheth L.U.J College of Arts & Sir M.V. College Of Science & Commerce

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/PRINCIPLES OF OPERATING SYSTEM/OS Practical No. 6 [10-08-2026]/Q4.py

--- Round Robin (Quantum = 2 ms) ---
PID  AT   BT   CT   TAT   WT
P1    0    5    7    7    2
P2    4    2    6    2    0
P3    5    4   11    6    2
Average TAT = 5.00 ms
Average WT  = 1.33 ms

--- FCFS ---
PID  AT   BT   CT   TAT   WT
P1    0    5    5    5    0
P2    4    2    7    3    1
P3    5    4   11    6    2
Average TAT = 4.67 ms
Average WT  = 1.00 ms
>>>
```

5. Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given Time Quantum = 2 ms. Calculate average turn around time and average waiting time.

Process	Arrival Time	Burst Time
P1	0 ms	5 ms
P2	1 ms	3 ms
P3	2 ms	6 ms

CODE:

```
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.rem = bt
        self.ct = 0
        self.tat = 0
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

self.wt = 0

```
def print_result(title, procs):
    print(f"\n--- {title} ---")
    print("PID\tAT\tBT\tCT\tTAT\tWT")
    total_tat = total_wt = 0
    for p in procs:
        print(f"P{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
        total_tat += p.tat
        total_wt += p.wt
    n = len(procs)
    print(f"Average TAT = {total_tat / n:.2f} ms")
    print(f"Average WT = {total_wt / n:.2f} ms")
```

```
def fcfs(procs):
    time = 0
    for p in procs:
        if time < p.at:
            time = p.at
        time += p.bt
        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt
    print_result("FCFS", procs)
```

```
def round_robin(procs, quantum):
    n = len(procs)
    queue = []
    arrived = [False] * n
    time = 0
    done = 0

    # enqueue processes already arrived at time 0
    for i, p in enumerate(procs):
        if p.at <= time and not arrived[i]:
            queue.append(i)
            arrived[i] = True
```

NAME: KRISHNA YADAV
ROLL NO: S124
SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

```
while done < n:
```

```
    if not queue:
```

```
        # jump to the next process that hasn't arrived yet
```

```
        next_i = min((i for i in range(n) if not arrived[i]), key=lambda i: procs[i].at)
```

```
        time = procs[next_i].at
```

```
        queue.append(next_i)
```

```
        arrived[next_i] = True
```

```
        continue
```

```
    idx = queue.pop(0)
```

```
    p = procs[idx]
```

```
    slice_time = min(p.rem, quantum)
```

```
    time += slice_time
```

```
    p.rem -= slice_time
```

```
for i, q in enumerate(procs):
```

```
    if q.at <= time and not arrived[i]:
```

```
        queue.append(i)
```

```
        arrived[i] = True
```

```
if p.rem > 0:
```

```
    queue.append(idx)
```

```
else:
```

```
    p.ct = time
```

```
    p.tat = p.ct - p.at
```

```
    p.wt = p.tat - p.bt
```

```
    done += 1
```

```
print_result(f"Round Robin (Quantum = {quantum} ms)", procs)
```

```
if __name__ == "__main__":
```

```
    quantum = 2
```

```
    p1 = [Process(1, 0, 5), Process(2, 1, 3), Process(3, 2, 6)]
```

```
    round_robin(p1, quantum)
```

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: PRINCIPAL OF OPERATING SYSTEM



Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir

M.V. College

Of Science & Commerce

OUTPUT:

```
===== RESTART: C:/Users/KRISHNA/Desktop/SYCS 27-28/PRINCIPLES OF OPERATING SYSTEM/OS Practical No. 6 [10-08-2026]/Q5.py =====
--- Round Robin (Quantum = 2 ms) ---
PID  AT   BT   CT   TAT  WT
P1    0    5   12   12    7
P2    1    3    9    8    5
P3    2    6   14   12    6
Average TAT = 10.67 ms
Average WT  = 6.00 ms
```

Ln: 31 Col: 0

NAME: KRISHNA YADAV

ROLL NO: S124

SUBJECT: PRINCIPAL OF OPERATING SYSTEM